ВАРИАНТ **19**

За курсов проект по дисциплина „Информационни системи”

Задание 19: Система за управление на автопарк

Описание на задачата: Създайте система за управление на автопарк на фирма, притежаваща множество превозни средства (напр. служебни автомобили, камиони). Системата трябва да поддържа информация за всяко превозно средство, график за поддръжка и ремонт, информация за разходите (гориво, сервиз) и данни за шофьорите, които ги ползват. Целта е по-добро планиране на поддръжката и разходите, както и следене на наличността на превозни средства.

Функционални изисквания: - Регистър на превозни средства: за всяко МПС се пазят данни – регистрационен номер, марка/модел, година, тип (лек автомобил, ван, камион и др.), пробег, текущо състояние (активно, в ремонт, спрян от движение). - План за поддръжка: въвеждане на периодични сервизни прегледи – напр. технически преглед на всеки 1 година, смяна на масло на всеки 10,000 км. Системата трябва да може да съхранява тези изисквания и да напомня, когато наближат (по пробег или по дата). - История на ремонти и разходи: запис на всяка извършена поддръжка/ремонт с дата, описание, цена, както и запис на текущия километраж тогава. Отделно регистриране на разходи за гориво (може опростено – запис на зареждания с дата, количество, цена). - Управление на шофьори (опционално): списък на шофьорите/служителите, които ползват МПС-тата, и връзка кой автомобил на кого е поверен (ако автомобилите се индивидуално назначават) или поне история кой е ползвал кой автомобил и кога. - Справки: график на предстоящи обслужвания (напр. списък кои МПС имат техн. преглед скоро), отчет на разходите по превозно средство за избран период, списък на неактивните МПС (например спрени поради повреда).

Насоки за изпълнение: - Проектирайте базата данни с таблици за превозни средства, планови дейности (тип дейност, интервал – км или дни), запис на конкретни извършени дейности (свързани с МПС), разходи за гориво, и шофьори. - За реализацията на напомняния: можете при стартиране на приложението или с бутон "Провери обслужване" да се изпълнява логика, която сравнява текущия пробег и дата със зададените интервали и маркира кои МПС-та се нуждаят от обслужване. Изведете ги в отделен списък или визуално маркирайте. - Интерфейсът трябва да улеснява въвеждането на нови данни: напр. при връщане на автомобил от курс, да може да се въведе нов пробег и евентуално заредено гориво; при завършен ремонт – да се въведе лесно нов запис за ремонт. - Обмислете удобство: напр. падащи менюта за типове дейности (масло, гуми, преглед) при въвеждане на ремонт, автоматично изчисляване колко дни остават до следващия тех. преглед, и т.н., за да се минимизира ръчната сметка. - Сигурност: ако системата е използвана от много потребители, може да се добавят роли – механик (въвежда ремонти), логистик (въвежда гориво и курсове), мениджър (преглежда отчети). Но това не е задължително за базов проект.

Критерии за оценка: - Системата дава цялостен поглед върху автопарка: налични МПС, техния статус, предстояща поддръжка. Нито една важна информация (напр. предстоящ преглед) не трябва да липсва от функционалността. - Данните са последователно съхранени: историята на всеки автомобил е пълна и може да се проследи (ремонти, разходи). Има целостност (напр. не може да се въведе ремонт за несъществуващ автомобил). - Справките/напомнянията реално биха помогнали в управление: например списък на коли над определен пробег без смяна на масло, или коли с изтичаща застраховка (ако включите застраховки) – тези демонстрират полезността на системата. - Интерфейсът е удобен за технически и нетехнически потребители: полетата са ясно обозначени, критични функции са достъпни но защитени (например триене на запис - с потвърждение).

Препоръчителни технологии/инструменти: - Може да реализирате като десктоп приложение (C# .NET, Java) ако предполагаме, че се ползва само в офиса, или като уеб приложение за достъп отвсякъде. - База данни: MySQL или SQLite, според това дали ще има паралелен достъп. SQLite е лесна за десктоп вариант. - Инструменти: API на Google Calendar или друг календар (опционално) за интегриране на напомняния; библиотека за графики (например Chart.js за web или OxyPlot за .NET) ако искате да покажете графично разходите във времето.

Стъпки за изграждане на ИС (основа на всички задания):

1. **Формиране на екипи / индивидуални задания** – всеки студент получава различна система.
2. **Проектиране** (моделиране – диаграми, архитектура)
3. **Събиране на изисквания** – кратко описание какво трябва да прави системата.
4. **Моделиране** –
    - Use Case диаграма (какви функции има)
    - ER диаграма на базата данни
    - Клас диаграма
5. **Създаване на база данни** – проектиране на таблици и връзки.
6. **Имплементация на CRUD операции** (create, read, update, delete).
7. **Изграждане на интерфейс** – web форма или desktop.
8. **Добавяне на проста бизнес логика** – напр. автоматично изчисление, валидиране на данни.
9. **Тестване** – въвеждане на примерни данни, проба за грешки.
10. **Документация** – кратко ръководство (Word/PDF или README.md).
11. **Презентация** – студентът представя проекта си.

Общи критерии за оценка:

- ✓ **Съответствие на проекта с оригиналната задача / спецификация**
- ✓ **Качество на кода:** четимост, модулност, коментари, спазване на добри практики
- ✓ **Обработка на грешки и устойчивост:** как се държи системата при неправилни входни данни, негативни сценарии
- ✓ **Потребителски интерфейс и потребителско изживяване (UI/UX)**
- ✓ **Документация:** как се инсталира, как се тества, потребителска документация + техническа документация
- ✓ **Тестове:** unit тестове, интеграционни тестове, поне няколко сценария – нормални и гранични случаи
- ✓ **Сигурност:** валидиране на вход, защита от SQL Injection / XSS / CSRF ако е web, защита на лични данни
- ✓ **Деплоймънт / използване на инфраструктура:** как е поставена системата, как се осигурява uptime, резервни копия